Real-time applications provide deterministic and deadline constrained responses. The Linux OS provides real-time support based on the POSIX standards 1003.b and 1003.c for running applications in real time. The real-time functions are found in the *librt* library, which must be included while running the POSIX synchronisation applications that are discussed. While compiling the programs, we must include the *–lrt* option to include the *librt* library to use real-time extensions provided by POSIX standards.

POSIX message queues are created in the Linux virtual file system; i.e., POSIX message queues are file based. This message queue file system is mounted onto the system by default or you can mount by using the following commands in super user or sudo mode:

```
$ mkdir /dev/mqueue
$ mount  -t mqueue none  /dev/mqueue
```

POSIX message queues have an associated structure called *mq_attr;* its members are shown in Figure 3. Message queues have attributes that can be accessed and set using the *mq_getsetattr* system call.

mq_flags: These are used to pass flags to the message queue. They can be used to block the message queue until messages arrive at the queue with the appropriate flags mentioned. It is set once the message queue is open using *mq_open*.

mq_maxmsg: This indicates the ceiling to the amount of messages that can be sent in the queue to the process.

mq_msgsize: The length of all the messages on the message queue in bytes is mentioned by this parameter.

mq_curmsgs: This returns the number of messages present in the message queue at that instant. When the queue is initialised, it is 0 and changes once it receives messages from the sending process or when the messages are eaten up by the receiving end.

The maximum number of messages that can be placed in the queue is 10 and the maximum size is 8192 bytes. This is mentioned in *proc/sys/fs/mqueue/msg_max* and *proc/sys/fs/mqueue/msgsize_max*.

Creating a POSIX message queue

The successful creation of a POSIX message queue returns a message queue descriptor. It returns *-1* if there is an error.

The *mq_open* system call is used to, as the name suggests, open a message queue. It accepts four parameters — the first is of type *char* to specify the name of the queue, the flag attribute of which can accept O_RDONLY for receiving messages, O_WRONLY for sending messages and O_RDWR to read and write messages to the message queue. O_CREAT is used along with the other flags to create a message queue if it doesn't already exist. The second parameter, O_NONBLOCK can be used to open the queue in non-blocking mode, where, by default the process blocks until it receives messages in the queue. The third parameter indicates the mode in which a process can access the message queue, i.e., the permissions that are given to a process to modify and alter the queue. The final parameter is a reference to the *mq_attr*.

Since the POSIX API allows the programmer to have control over the geometry of the queue, the attribute members of the structure *mq_attr* can be filled according to user requirement. However, attributes such as *mq_maxmsg* and *mq_msgsize* must be constrained to the values mentioned in */proc/sys/fs/mqueue/msg_max* and *proc/sys/fs/mqueue/msgsize_max*. The message queue attribute is a structure that consists of four elements of type long integer — the flags for the queue, the ceiling amount of messages that can be allowed in the queue, the ceiling size of the message in bytes, and the number of messages available in the queue.

Here, we declare *mq_create* of type *mqd_t*, which is the message queue descriptor, and *mq_ret* and *i* of type *int* to test the return values of the *mq_open* call. We fill the attribute values of the *mq_attr*, setting the maximum messages in the queue as 10 and the maximum size of the messages as 8192 bytes.

In *mq_open* we pass the parameters — message queue name (*/mymq*, which needs to begin with a "/"); the flags O_CREAT (which creates the message queue if it doesn't already exist) and O_RDWR to read and write from and to the message queue; the access permission and, finally, the queue attributes.

If the *mq_open* call returns successfully with a valid queue descriptor value, we print the message: "Message Q is successfully created ...."; otherwise, we print "Message Q is

```
struct mq_attr {
    long mq_flags;       /* Flags: 0 or O_NONBLOCK */
    long mq_maxmsg;      /* Max. # of messages on queue */
    long mq_msgsize;     /* Max. message size (bytes) */
    long mq_curmsgs;     /* # of messages currently in queue */
};
```

Figure 3: Attribute structure of POSIX message queues

```
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

int main ( ) {

        mqd_t mq_create;
        int mq_ret, i;
        i=-1;
        struct mq_attr attr;
        attr.mq_flags=0;
        attr.mq_maxmsg=10;
        attr.mq_msgsize=8192;
        attr.mq_curmsgs=0;

        mq_create=mq_open("/mymq", O_CREAT|O_RDWR, 0744, &attr);
        if(mq_create == 3)
                printf (" Message Q is successfully created ....\n");
        else
                printf (" Message Q is not created....\n");
}
```

Figure 4: Creating message queues in POSIX